Open Vision / ML Engineer Intern – 72-Hour Technical Challenge

Overview

You have been provided with:

- Three high-resolution aerial imagery datasets
- Corresponding feature layers / annotations
- Feature definitions and class labels

Your objective is to develop a machine learning solution capable of learning from the provided imagery and identifying the same features on previously unseen aerial imagery.

The final solution must be reproducible and executable by the Ottermap team.

Challenge Objective

Build an end-to-end computer vision pipeline that:

1. Uses the provided imagery and labels for training.
2. Learns to identify mapped features.
3. Generalizes to new aerial imagery not included in the training set.
4. Produces GIS-compatible outputs.
5. Can be executed by the Ottermap team on unseen imagery for evaluation.

Features

Depending on the supplied dataset, features may include:

- Turf / Grass

You may use object detection, semantic segmentation, instance segmentation, foundation models, or hybrid approaches.

Requirements

1. Model Development

You may use any open-source tools, frameworks, or pre-trained models, including:

- PyTorch
- TensorFlow
- Detectron2
- OpenMMLab
- Ultralytics
- Hugging Face

Examples:

- YOLO
- U-Net
- DeepLab
- SegFormer
- Mask R-CNN
- SAM
- Grounding DINO

Training from scratch is not required.

Transfer learning and foundation models are encouraged.

---

2. Generalization Test

In addition to the provided imagery:

- Source at least one additional aerial imagery dataset from a different geographic location.
- Run your model on this unseen imagery.
- Demonstrate the model's ability to identify features outside the training dataset.

Document any successes, limitations, and failure cases.

---

3. GIS Output Generation

Generate GIS-compatible outputs for model predictions.

Accepted formats:

- GeoJSON

- Shapefile
- Polygon masks

Predictions should be suitable for downstream GIS workflows.

Deliverables

A. GitHub Repository

Provide a repository containing:

- Source code
- Environment setup instructions
- Dependencies
- Training workflow
- Inference workflow

B. Trained Model

Provide:

- Model weights
- Configuration files
- Checkpoints

Accepted formats:

- .pt
- .pth
- .onnx
- .ckpt

C. Inference Script

Provide a simple command that allows evaluation on new imagery.

Example:

python inference.py –image input_image.tif

or

python inference.py –input ./images/

The Ottermap team will run this script against unseen imagery during evaluation.

D. Results Folder

Include:

1. Training imagery predictions
2. Validation imagery predictions
3. Predictions on externally sourced imagery

Provide visual overlays showing identified features.

E. GIS Outputs

Provide sample outputs including:

- GeoJSON files
- Polygonized detections
- Vector outputs

F. Technical Summary (Maximum 3 Pages)

Include:

Approach

- Model architecture
- Training methodology

Dataset Preparation

- Preprocessing
- Augmentation

Results

- Observed accuracy
- Examples of successes and failures

Improvements

- What would be done with additional time or data

Evaluation Criteria

Model Performance & Generalization (40%)

Ability to identify features on imagery not used during training.

Engineering Quality (20%)

Code quality, reproducibility, and documentation.

GIS Output Quality (15%)

Accuracy and usability of generated vector outputs.

Technical Reasoning (15%)

Choice of architecture, experimentation, and problem-solving.

Innovation (10%)

Use of modern computer vision techniques and efficient workflows.

Submission

Submit:

1. GitHub Repository Link
2. Model Weights
3. Technical Summary (PDF)
4. Sample Outputs

via a shared Google Drive folder or GitHub repository.

Timeline

Challenge Release: Day 0

Submission Deadline: 72 Hours from Receipt of Assignment

Estimated Effort: 10–20 Hours

Late submissions will not be evaluated.